Appendix I — GitLab interview question bank

Source: `GITLAB_INTERVIEW_QUESTION_BANK.md`

GitLab — Karan Bhatia: Interview Question Bank (current as of 2026-10-02)

This is a **synthesis of everything confirmed as of today** — a tighter, question-and-answer companion to `GITLAB_KARAN_BHATIA_DEEP_DIVE.md` (the full 80-page reference) and `GITLAB_WEEK_PLAN.md` (the day-by-day schedule). It exists because today's session surfaced the **real** Agentforce for Setup v2 architecture (from the actual HLD and an Agent-Runtime teaching guide), which is sharper and more defensible than anything reconstructed earlier. Use this doc to rehearse; use the probes (`salesforce_architecture_probe.md`, `OFFICE_CLAUDE_QUESTIONS.md`) to close the PENDING items before Oct 9.

Status flags used throughout:

- **READY** — grounded in a real source document, safe to answer with confidence today.
- **PENDING** — the mechanism is known, but the specific number/citation needs an office-Claude answer before you should state it as fact in the room.
- **HONEST GAP** — explicitly not built. Say so plainly; don't improvise.

Interview: **Friday, 2026-10-09**, with **Karan Bhatia** (VP Engineering & Engineering Fellow, GitLab — joined ~Sep 2026; previously VP Eng & Technical Fellow / Senior Principal Architect at New Relic, 5.5 years in Azure AD/IAM at Microsoft before that). Role: **Senior Software Engineer, Agentic SDLC**, building agents that author MRs, generate tests, and remediate security findings inside GitLab's Duo Agent Platform.

---

1. Your three bridges to him — lead with these, deliberately

1. **Identity & multi-tenancy.** 5.5 years of his career was Azure AD/IAM at Microsoft; his self-described New Relic specialty is "scaling multi-tenanted systems." Your strongest material: the four-stage permission gate below (§5), and your Salesforce "runs under the requesting user's own permissions" design.
2. **Observability as agent correctness.** He personally wrote a blog post on distributed tracing (`traceId`/`tenantId`/`upstreamApplication` correlation attributes) and champtogether a New Relic × GitHub Copilot closed-loop (observability signal → auto-filed issue → agent drafts a fix/PR → human review → post-merge validation) — structurally identical to what this role does. Your eval frameworks and ResolveFlow's diagnose→propose→approve→verify loop are the same shape.
3. **He's ~2 months into GitLab.** Don't assume he knows GitLab's Rails internals better than you do yet. He'll probe general principles and your own systems more than GitLab trivia.

---

2. Agentforce for Setup v2 — the real architecture, Q&A

Q: "Walk me through an agent architecture you've built."

> READY. "It's a Supervisor/Specialists model. A persistent `ConversationManager` triages every request into `INFORMATION_SEEKING` or `ACTION_SEEKING`, then delegates to a `Knowledge Node` (retrieval — docs, NLDs, live metadata grounding) or a `Reasoner Node` (planning). The Reasoner either takes a High-Velocity Path — a single matching workflow doc translated directly into an execution plan — or a Dynamic Synthesis Path for anything more complex, synthesizing multiple workflow/best-practice documents into one plan. An `Executor` node then

> runs the plan with zero reasoning — pure deterministic code, 100% fidelity, no LLM in the execution path at all."

Q: "How do you keep tool/action selection accurate as your catalog grows to hundreds of tools?"

> READY — this is your strongest mechanism answer. "A two-stage funnel. Stage one: for each step in the plan scaffold, a parallel vector search narrows the full action catalog down to 3–5 relevant candidates — that's RAG applied to tool discovery, not document retrieval. Stage two: one grounded LLM call picks the best of that small candidate list per step and emits the full structured plan. The LLM never has to search hundreds of options — it only ever reasons over five. That's what keeps it fast and accurate as the catalog scales, instead of degrading."

Q: "What happens when the agent doesn't have enough information to act?"

> READY. "A dedicated clarification phase runs before any plan is compiled — it checks the goal against the documented requirements for that operation, and if something critical is missing it pauses and asks the user directly, rather than guessing. Only once requirements are satisfied does the deterministic scaffold get extracted and compiled into a plan."

Q: "How do you guarantee an agent's actions are predictable once a plan is approved?"

> READY. "The execution layer contains no reasoning at all — it's plain deterministic code that runs each step of the approved plan exactly as specified, resolves data dependencies between steps (an ID produced in step one feeding step three), and never deviates. Separating 'thinking' from 'doing' into two different components, not just two phases of one component, is the core reliability guarantee."

Q: "What's the single hardest trade-off in this design?"

> READY (HLD-sourced, volunteer this if asked for weaknesses — it reads as self-awareness, not a confession). "The new stateful planning architecture is measurably slower per-request than the simpler reactive planner it replaced — roughly 10-20% at the time this was being rolled out — because real planning and clarification cost more than picking the most-likely action. The bet is that reliability and the ability to handle the long tail of cases is worth that latency, and that platform-level investment closes the gap over time. I'd rather state that trade-off explicitly than pretend the new architecture is free."

---

3. V1 → V2 migration — likely questions

Q: "Why did this move from v1 to v2, technically — not a feature list?"

> READY. Four concrete limits of v1, one sentence each: a reactive planner with no real reasoning step; no clarification capability, so it just picked the most likely action and lived with being wrong sometimes; LLM-based routing that stopped scaling once the action catalog passed a few dozen entries; and capability growth required writing new backend code for every addition. v2's answer to each, paired one-to-one: real multi-step planning, an explicit clarification loop, vector-search-driven action discovery instead of routing logic, and "knowledge as code" — adding a capability means writing a structured document, not new code.

Q: "How did you avoid a risky big-bang rewrite?"

> READY. "Co-existence behind a permission gate — the new architecture ran alongside the old one, workflow by workflow, with old configuration converted into the new document format rather than thrown away, and legacy bespoke actions kept working while they were incrementally replaced by generic ones. Nothing was cut over in one shot."

Q: "What was YOUR specific role in that migration — which workflows, how many documents did you personally author?"

> **PENDING** — needs `salesforce_architecture_probe.md` §3. Don't answer with a company-wide number as if it's personal share. Get the real count before Oct 9.

---

4. Data Cloud retrieval — likely questions

Q: "How does retrieval/grounding actually work end to end?"

> READY (mechanism). "A request is authenticated through a token-exchange step that mints a scoped, per-org identity token, routed through a central AI gateway into the data platform, where two retrievers run — one against a general knowledge/help index, and a separate one against an index built specifically from the structured operational documents that describe how to perform each task. Results flow back through the same gateway to the planner." Draw the five-hop chain if asked to whiteboard it.

Q: "What's the latency through that pipeline, and what happens on partial failure?"

> **PENDING** — needs `salesforce_architecture_probe.md` §2. Have the shape of the answer ready (name each hop, say you'd check X dashboard) even if the exact number isn't filled in yet.

Q: "How do you handle staleness — an org's data changes, how long until the agent sees it?"

> READY (mechanism) / **PENDING** (real number). "Ingestion runs as a staged pipeline with its own status machine — queued, processing, syncing, copied, then ingestion requested and complete — rather than being instantaneous. There is a real lag between a source change and the agent seeing it; I don't want to quote a number I haven't re-verified recently, but the mechanism is a periodic extraction-and-sync job, not real-time streaming." Get the real cadence from the probe.

---

5. The permission/security gate — your strongest section, lead with it unprompted

Q: "How do you make sure an agent can't do something it shouldn't?"

> READY — this is the answer that "makes an interviewer sit up." Don't merge it into one fuzzy check; state it as four independent, sequential gates: 1. **Grounding** — is this real? Do the referenced entities actually exist, is the request coherent against live system state. Catches hallucinated or malformed plans. 2. **Permission** — is *this caller* allowed *this operation* on *this specific resource*? Checked against roles/policies/allowlists. Catches unauthorized-but-well-formed requests. 3. **Approval** — is this action high-impact enough to need a human? If so, pause and wait for explicit sign-off before anything executes. 4. **Contract** — is the call shaped exactly as

> the tool's schema defines, independent of whether the caller was authorized? Catches malformed calls even from authorized callers.
>
> "The reason to keep these four separate instead of one fuzzy check: a plan can pass one and fail another. A well-formed request to grant a real user admin access that this caller has no right to grant passes grounding and fails permission. A request to grant access to a hallucinated user that doesn't exist would pass permission and fail grounding. One check can't catch both failure modes — that's the whole argument for separating them."

Q: "What's the evidence this actually works, not just design intent?"

> READY. "A measurable ~35% reduction in misconfigured access requests, directly attributable to schema validation and operation-level allowlists catching bad calls before they land — that's the permission and contract stages doing their job in production, not just on paper."

---

6. Honest gaps — say these plainly, don't bluff

Q: "How does a session survive a crash — can an agent resume a multi-step task after a restart?"

> **HONEST GAP.** "I can reason clearly about how I'd build that — periodic state snapshots as the resume source, with an append-only event log kept separately as an audit trail rather than the replay mechanism, because replaying tool-call events on resume risks re-triggering real side effects like re-granting access. But durable crash recovery isn't something I've shipped myself — it's a pattern I've studied, not built. I'd rather tell you that directly than describe a system as if I ran it in production."

Q: "How do you handle a session that runs long enough to exceed the model's context window?"

> **HONEST GAP**, but the reasoning bridges cleanly from real experience. "The core trade-off is the same one I already navigate in retrieval work — deciding what to keep in full detail versus what to compress, and the risk is that a specific fact compressed too early is effectively lost unless it's written somewhere durable it can be pulled back from. I haven't built a compaction system specifically, but it's the same completeness-versus-budget tension as RAG, just applied to conversation history instead of documents."

---

7. GitLab-specific bridges — talking points, not just answers

- If he brings up GitLab's own vision for agents committing/opening MRs "at machine rate" with governance as a default platform service: connect it directly to your permission-gate framework (§5) — the same four-stage shape applies whether the agent is modifying Salesforce access policies or pushing code.
- If observability/debugging comes up: propose propagating a `traceId`/session id plus `tenantId`/`requestingUserId` through every node transition and tool call in an agent graph — his own correlation-attribute pattern, applied to agent execution instead of HTTP requests.
- A good question to ask **him**, given he's new to GitLab: "*What does your team currently own on the Duo Agent Platform roadmap, and where's the biggest gap between the public vision and what's actually shipped today?*" — genuine, calibrates the conversation, and is the kind of question a fast-ramping VP appreciates being asked.

---

8. Deployment / infra — cross-reference, mostly PENDING

This axis cost you the CrowdStrike round — do not shortchange it here. Full prompts are in `OFFICE_CLAUDE_QUESTIONS.md` (sections A–F: AWS architecture, CI/CD, Kubernetes, Splunk/ observability with a real incident, Data Cloud infra specifics, the "~40% throughput" and on-call claims). Until those come back: **don't state specific AWS service choices, CI/CD stages, or the 40% figure's methodology as fact.** The honest fallback if asked before you've verified: "I can describe the pattern — containerized, orchestrated for fault isolation, event-driven — but let me get you the specific configuration rather than guess from memory."

---

9. Consolidated likely-question list, by difficulty

Foundational

- What's the difference between an agent "deciding" to do something and it actually happening?
- Why keep grounding and permission as separate checks instead of one validation step?
- What's the difference between information-seeking and action-seeking requests, and why classify before planning?

Senior

- Walk me through what happens, step by step, from a user's request to a real side effect.
- Why does your execution layer have zero reasoning in it — what would go wrong if it had some?
- How do you keep an LLM's tool-selection accurate as the tool catalog grows to hundreds of tools?
- What's a production failure you've debugged in one of these systems, from symptom to fix? *(PENDING — §7 of the probe needs a real incident.)*

Staff

- Two concurrent requests touch the same underlying record — what happens, and how do you make the system correct without relying on the model to coordinate? *(Not directly covered by the HLD — reason about it live using your Salesforce permission-gate + ResolveFlow idempotency experience.)*
- Compare bespoke, hand-written tools against generic, broadly-parameterized tools — when is each right, and how does the security analysis change? *(READY — v1's 45 bespoke actions vs. v2's generic CRUD actions is a real, lived example of exactly this trade-off.)*
- Design the eval framework for an agent that generates tests or drafts MRs. *(Bridge from your Salesforce eval framework and ResolveFlow's 50-scenario suite — regression, routing accuracy, permission enforcement, cost/latency, as categories; see DD §9 in the full deep-dive.)*

Fellow/VP-level

- If this agent is fully compromised by an attacker right now, what's the worst thing it can actually cause? Walk the four-stage gate and show the blast radius is bounded at each layer.
- You're new to a team and inherit an agent platform with known, documented risks (slower than the system it replaced, a retrieval pipeline whose scalability is "still uncertain," ongoing parity gaps in the new runtime). How do you prioritize what to fix first? *(This is close to Karan's own situation at GitLab right now — answering it thoughtfully doubles as showing you understand his position.)*

---

10. Before Oct 9 — what must move from PENDING to READY

1. Your actual share of the V1→V2 migration (§3) — real NLD count, real workflow list.
2. Data Cloud retrieval latency per hop, and real ingestion-lag cadence (§4).
3. One real production incident, start to finish, from memory (§9 Senior tier).

4. The AWS/CI-CD/K8s/Splunk/"40% throughput" items in `OFFICE_CLAUDE_QUESTIONS.md` — these are the single highest-risk category given what happened at CrowdStrike. Don't leave them for Oct 8.
